

LOGIC FORMULATION AND INTRODUCTORY PROGRAMMING

WHILE AND FOR LOOPS

Introduction to Repetition Structures

- Often have to write code that performs the same task multiple times
- Disadvantages to duplicating code
 - ★ Makes program large
 - ★ Time consuming
 - ★ May need to be corrected in many places
- **Repetition structure:** makes computer repeat included code as necessary
- Includes condition-controlled loops and count-controlled loops

The while Loop: A Condition-Controlled Loop

Condition tested for true or false value
Statements repeated as long as condition is true

- The while loop starts with while and a Boolean condition, followed by a colon.
- In order for a loop to stop executing, something has to happen inside the loop to make the condition false
- **iteration:** one execution of the body of a loop
- while loop is known as a pretest loop
- it tests its condition before performing an iteration
- you usually have to perform some steps prior to the loop to make sure that the loop executes at least once.

Infinite Loops

- loop that does not have a way of stopping
- Repeats until program is interrupted
 - Occurs when programmer forgets to include stopping code in the loop

The for Loop: A Count-Controlled Loop

- iterates a specific number of times
- Use a for statement to write count-controlled loop items
- Designed to work with sequence of data

- the for keyword opens the for loop (there's no condition after it)
- any variable after the for keyword is the control variable of the loop; it counts the loop's turns, and does it automatically

Using the range Function with the for Loop

The range function simplifies the process of writing a for loop

- ★ range returns an iterable object
- ☆ **iterable** - contains a sequence of values that can be iterated over

- ☆ In the different For statements, different arguments are passed to the range function:
- ☆ **1 argument (5):** used as ending limit
- ☆ **2 arguments (1, 5):** starting value and ending limit
- ☆ **3 arguments (1, 10, 2):** starting value, ending limit, and the step value
- ☆ Be sure to consider the end cases: range does not include the ending limit
- ☆ The range function can be used to generate a sequence with numbers in descending order
 - ★ Make sure starting number is larger than end limit, and step value is negative

Using the Target Variable Inside the Loop

- ☆ **Target variable:** the variable which is the target of the assignment at the beginning of each iteration
- ☆ to reference each item in a sequence as the loop iterates
- ☆ can be used in calculations or tasks in the body of the loop

The Augmented Assignment Operators

- ☆ Example: $x = x + 1$

Shorthand operators:

+=	-=	
*=	/=	%=

SENTINEL AND INPUT VALIDATION

- A type of loop that continues to execute until it encounters a special value — called the sentinel value — which signals the loop to stop. This is useful when you don't know ahead of time how many inputs you will need, such as when reading input from a user until they choose to quit.
- A **sentinel value** is a specific value used to terminate a loop. It must be a value that is not valid in the normal input set so the program can recognize it as a stopping signal.

Input Validation is the process of ensuring that user-provided data is correct, meaningful, and safe to use before processing it. This is important to prevent errors, unexpected behavior, or security issues in your programs.

- Python provides various techniques for validating input, such as checking data types, using loops to repeatedly ask for valid input, or applying conditions.
- *Why Validate Input?*
 - ★ To prevent runtime errors (converting a string to an integer when it contains letters).
 - ★ To ensure data integrity and logic correctness.
 - ★ To guide users to enter data in the expected format.

NESTED LOOPS

Nested loops allow you to work with complex data structures and perform operations on multiple dimensions. They enable you to solve problems that require iteration over multiple levels

- **Example:** analog clock works like a nested loop

Key points about nested loops:

- ★ Inner loop goes through all of its iterations for each iteration of outer loop

- ★ Inner loops complete their iterations faster than outer loops
- ★ Total number of iterations in nested loop = $\text{number_iterations_inner} \times \text{number_iterations_outer}$

FUNCTIONS

- group of statements within a program that perform as specific task
- Usually one task of a large program
- Functions can be executed in order to perform overall program task
- Known as divide and conquer approach
- **Modularized program:** program wherein each task within the program is in its own function

Benefits of Modularizing a Program with Functions

- ★ Simpler code
- ★ Code reuse - write the code once and call it multiple times
- ★ Can test and debug each function individually
- ★ Faster development
- ★ Different team members can write different functions

Void Functions and Value-Returning Functions

- **A void function:** Simply executes the statements it contains and then terminates.
- **A value-returning function:**
 - ★ Executes the statements it contains, and then it returns a value back to the statement that called it.
 - ★ The input, int, and float functions are examples of value-returning functions.

Defining and Calling a Function

- Function name should be descriptive of the task carried out by the function. Often includes a verb
- **Function definition:** specifies what function does
- **Function header:** first line of function
 - ★ Includes keyword def and function name, followed by parentheses and colon

- When a function is called:
 - ★ Interpreter jumps to the function and executes statements in the block
 - ★ Interpreter jumps back to part of program that called the function, known as function return
- **main function:** called when the program starts
 - ★ Calls other functions when they are needed
 - ★ Defines the mainline logic of the program

Designing a Program to Use Functions

- In a flowchart, function call shown as rectangle with vertical bars at each side
 - ★ Function name written in the symbol
 - ★ Typically draw separate flow chart for each function in the program
- End terminal symbol usually reads Return
- **Top-down design:** technique for breaking algorithm into functions
- **Hierarchy chart:** depicts relationship between functions
 - ★ structure chart
 - ★ Box for each function in the program, Lines connecting boxes illustrate the functions called by each function
 - ★ Does not show steps taken inside a function
- Use input function to have program wait for user to press enter

Local Variables

- Variable that is assigned a value inside a function
 - ★ Belongs to the function in which it was created
 - ★ Only statements inside that function can access it, error will occur if another function tries to access the variable
- **Scope:** the part of a program in which a variable may be accessed
 - ★ For local variable: function in which created
 - ★ Local variable cannot be accessed by statements inside its function which precede its creation

- ★ Different functions may have local variables with the same name

Passing Arguments to Functions

- **Argument:** piece of data that is sent into a function
 - ★ Function can use argument in calculations
 - ★ When calling the function, the argument is placed in parentheses following the function name
- **Parameter variable:** variable that is assigned the value of an argument when the function is called
 - ★ The parameter and the argument reference the same value
- **Scope of a parameter:** the function in which the parameter is used

Passing Multiple Arguments

- Parameter list replaces single parameter
- Parameter list items separated by comma
- Arguments are passed by position to corresponding parameters

Making Changes to Parameters

- Changes made to a parameter value within the function do not affect the argument
- Known as pass by value
- Provides a way for unidirectional communication between one function and another function
- Calling function can communicate with called function

Keyword Arguments

- argument that specifies which parameter the value should be passed to
- Position when calling function is irrelevant
- **General Format:**
Function name(parameter=value)
- Possible to mix keyword and positional arguments when calling a function
- Positional arguments must appear first

Global Variables and Global Constants

- created by assignment statement written outside all the functions

- Can be accessed by any statement in the program file, including from within a function
- **General format:** global variable_name

Reasons to avoid using global variables

- Global variables making debugging difficult
- Many locations in the code could be causing a wrong variable value
- Functions that use global variables are usually dependent on those variables
- Makes function hard to transfer to another program
- Global variables make a program hard to understand

Global Constants

- Global name that references a value that cannot be changed
- Permissible to use global constants in a program
- To simulate global constant in Python, create global variable and do not re-declare it within functions

Standard Library Functions and the import Statement

- **Standard library:** library of pre-written functions that comes with Python
 - ★ Library functions perform tasks that programmers commonly need
 - ★ Viewed by programmers as a "black box"
- **Modules:** files that stores functions of the standard library
- Help organize library functions not built into the interpreter
- Copied to computer when you install Python
- To call a function stored in a module, need to write an import statement
 - ★ Written at the top of the program
 - ★ **Format:** import module_name

Dot (.) notation

- for calling a function belonging to a module
- **Format:** module_name.function_name()

Writing Your Own Value-Returning Functions

- **Format:** return expression
 - ★ The value for expression will be returned to the part of the program that called the function
 - ★ The expression in the return statement can be a complex expression, such as a sum of two variables or the result of another value- returning function

How to Use Value-Returning Functions

- Value-returning function can be useful in specific situations
 - ★ Example: have function prompt user for input and return the user's input
 - ★ Simplify mathematical expressions
 - ★ Complex calculations that need to be repeated throughout the program
- Use the returned value
 - ★ Assign it to a variable or use as an argument in another function

Returning Boolean Values

- ◇ Boolean function: returns either True or False
- ◇ Use to test a condition such as for decision and repetition structures
- ◇ Use to simplify complex input validation code

Returning Multiple Values

- ◇ In Python, a function can return multiple values
 - ★ Specified after the return statement separated by commas
- ◇ **Format:** return expression1,
- ◇ When you call such a function in an assignment statement, you need a separate variable on the left side of the = operator to receive each returned value

The math Module

- ◇ part of standard library that contains functions that are useful for performing mathematical calculations
- ◇ Typically accept one or more values as arguments, perform mathematical operation, and return the result

- ✧ Use of module requires an import math statement
- ✧ The math module defines variables pi and e, which are assigned the mathematical values for pi and e
- ✧ **Example:** `circle_area = math.pi * radius**2`

Storing Functions in a Module

- ✧ In large, complex programs, it is important to keep code organized
- ✧ **Modularization:** grouping related functions in modules
- ✧ Makes program easier to understand, test, and maintain
- ✧ Make it easier to reuse code for multiple different programs
- ✧ Import the module containing the required function to each program that needs it
- ✧ Module is a file that contains Python code
- ✧ Contains function definition but does not contain calls to the functions
- ✧ Importing programs will call the functions
- ✧ Rules for module names:
 - ★ File name should end in .py
 - ★ Cannot be the same as a Python keyword
 - ★ Import module using import statement

Functions With No Parameters but With Return Values

- ✧ They do not require any arguments to be passed when called. However, these functions can still perform computations and return results to the calling code

Functions With Parameters and No Return Values

- ✧ Functions that accept arguments but do not return any value are commonly used when we need to perform a specific action or task without requiring a result to be returned. These functions are known as "void functions" because they do not return any value explicitly.

Functions With Parameters and Return Values

- ✧ widely used to perform specific operations and produce results that can be utilized in other parts of the program. These functions enable you to pass data to them, process it, and then return the computed result back to the calling code.

STRING OPERATIONS

strings are sequences of characters enclosed in single (' ') or double (" ") quotes. They are widely used for storing and manipulating text-based data.

strings are immutable sequences of Unicode characters. This means once defined, the contents of a string cannot be changed. To modify a string, you need to create a new string.

String Input

the process of obtaining a sequence of characters (a string) from a user or an input source. In Python, you can use the input() function to read user input as a string.

String Concatenation

the process of combining two or more strings into a single string. In Python, you can concatenate strings using the + operator or by using formatted string literals (f-strings). *Here's an example:*
`concatenated_str = str1 + str2`

String Comparison

Used to determine the order or equality of two strings. In Python, you can compare strings using comparison operators such as ==, !=, <, >, <=, and >=. These operators return True or False based on the comparison result. *Here's an example:*
`result = str1 == str2`

String Copying

Involves copying the contents of one string to another. In Python, since strings are immutable, you can create a copy of a string simply by assigning it to a new variable. *Here's an example:*
`str1 = "Hello"`
`str2 = str1`

Pre-defined String Functions

Python provides a set of built-in string methods that allow you to perform various operations on strings. These methods simplify string manipulation, searching, and other common tasks.

Standard String Methods

len() - used to determine the length of a string. It returns the number of characters in the string.

Here's an example:

```
str = "Hello, World!"  
length = len(str)
```

str.replace() - replace() method is used to replace occurrences of a substring in a string with a new substring. It returns a new string with the replacements made.

Here's an example:

```
str = "Hello, World!"  
new_str = str.replace("World", "Python")
```

str.strip() - strip() method is used to remove leading and trailing whitespace characters (such as spaces and tabs) from a string. It returns a new string with the whitespace removed.

Here's an example:

```
str = " Hello, World! "  
new_str = str.strip()
```

str.join() - join() method is used to join elements of an iterable (such as a list) into a single string. It returns a new string with the elements joined by the specified separator.

Here's an example:

```
words = ["Hello", "World", "Python"]  
sentence = " ".join(words)
```

str.startswith() and **str.endswith()** - used to check if a string starts or ends with a specified prefix or suffix, respectively. They return True or False based on the check.

Here's an example:

```
str = "Hello, World!"  
starts_with_hello = str.startswith("Hello")  
ends_with_world = str.endswith("World")
```

String Searching Operations

str.find() and **str.rfind()** - *find()* method is used to find the first occurrence of a substring in a string. It returns the index of the first occurrence or -1 if the substring is not found. The *rfind()* method works similarly but finds the last occurrence of the substring instead.

str.index() and **str.rindex()** - *index()* method is used to find the first occurrence of a substring in a string. It returns the index of the first occurrence or raises a `ValueError` if the substring is not found. The *rindex()* method works similarly but finds the last occurrence of the substring instead.

